MODULE 07 | SHELL SCRIPTING

Shell Scripting

Turn repeated commands into reusable programs with variables, conditionals, loops and functions.

Mission	Topic	Commands / Concepts
1	Variables and Input	variables, $(), read, $1 $@ $#, $?, quoting
2	Decisions: if, test and case	if, [ ], [[ ]], elif, case, &&, ||
3	Repeat: for, while and until	for, while, until, break, continue, seq
4	Reuse Code: Functions and Arrays	function, local, return, arrays, $(), source
5	Real Scripts: Exit Codes and Safety	exit, set -euo pipefail, trap, getopts, cron-ready scripts
Boss	The Automation Master	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

What is a shell script?

A text file of commands that Bash runs top to bottom. Anything you can type in the terminal you can put in a script and run again with one command.

Your first script

```
$ nano hello.sh
#!/bin/bash
# My first script
echo "Hello, $USER! Today is $(date +%A)."

$ chmod +x hello.sh
$ ./hello.sh
Hello, haani! Today is Saturday.
```

- `#!/bin/bash` (the **shebang**) tells the system which interpreter to use.
- `#` starts a comment.
- `chmod +x` makes the file executable (Module 04).
- `./hello.sh` runs it. You can also run `bash hello.sh` without the execute bit.

Good habits from day one

- Always quote variables: `"$name"`, not `$name`.
- Use meaningful names and comments.
- Test on sample data before pointing a script at real files.
- Run `bash -n script.sh` to check syntax and `bash -x script.sh` to trace each line.

MISSION 1 | +50 XP

Variables and Input

Goal: Store values, read user input and use script arguments.

```
name="Haani"            # no spaces around =
age=21
echo "Name: $name, Age: $age"
echo "Safe form: ${name}_file"     # braces separate the name from text

today=$(date +%F)       # command substitution
echo "Date: $today"
count=$(ls | wc -l)
echo "Files here: $count"
```

Quoting

Style	Behaviour
`"double"`	Variables and `$(...)` are expanded
`'single'`	Everything literal, nothing expands
`\`	Escapes one character

```
$ echo "Home is $HOME"     # Home is /home/haani
$ echo 'Home is $HOME'     # Home is $HOME
```

Reading input

```
read -p "Your name: " name
read -s -p "Password: " pw; echo       # -s hides typing
echo "Welcome, $name"
```

Arguments and special variables

Variable	Meaning
`$0`	Script name
`$1 $2 ...`	Positional arguments
`$#`	Number of arguments
`$@`	All arguments (each separate, quote it)
`$?`	Exit status of the last command (0 = success)
`$$`	PID of the script
`$USER $HOME $PWD`	Built-in environment variables

```
#!/bin/bash
# greet.sh
echo "Script: $0"
echo "First: $1, count: $#"
echo "All: $@"

$ ./greet.sh alice bob
Script: ./greet.sh
First: alice, count: 2
All: alice bob
```

Arithmetic and defaults

```
a=7; b=3
echo $((a + b))          # 10
echo $((a * b))          # 21
echo $((a % b))          # 1  (modulo)
name=${1:-World}         # default when $1 is empty
export PATH="$PATH:$HOME/bin"   # export makes a variable visible to child processes
```

Practice

- Write a script that asks for your name and age and prints a sentence.
- Make a script that takes two numbers as arguments and prints their sum and product.
- Print the number of files in the current folder using command substitution.

MISSION 2 | +50 XP

Decisions: if, test and case

Goal: Make scripts react to conditions.

```
if [ "$age" -ge 18 ]; then
    echo "Adult"
elif [ "$age" -ge 13 ]; then
    echo "Teen"
else
    echo "Child"
fi
```

Spaces inside `[ ]` are required: `[ "$a" = "$b" ]`, not `["$a"="$b"]`.

Number test	Meaning
`-eq -ne`	Equal, not equal
`-gt -ge`	Greater than, greater or equal
`-lt -le`	Less than, less or equal

String test	Meaning
`= !=`	Equal, not equal
`-z "$s"`	String is empty
`-n "$s"`	String is not empty
`[[ $s == a* ]]`	Pattern match (Bash)
`[[ $s =~ ^[0-9]+$ ]]`	Regex match (Bash)

File test	Meaning
`-e f`	Exists
`-f f`	Is a regular file
`-d f`	Is a directory
`-r -w -x f`	Readable / writable / executable
`-s f`	Exists and is not empty

```
if [ -f "$1" ] && [ -r "$1" ]; then
    echo "$1 is a readable file"
else
    echo "Cannot read $1" >&2      # message to stderr
    exit 1
fi

[ -d backup ] || mkdir backup      # create only if missing
grep -q root /etc/passwd && echo "root exists"
```

case - many choices

```
read -p "Action (start|stop|status): " act
case "$act" in
    start)   echo "Starting..." ;;
    stop)    echo "Stopping..." ;;
    status|s) echo "Running" ;;
    *)       echo "Unknown: $act" ;;
esac
```

Tip
Use `[[ ... ]]` in Bash scripts. It handles empty variables and patterns more safely than `[ ... ]`. Use `(( ... ))` for number comparisons: `if (( age >= 18 ))`.

Practice

- Write a script that tells whether a number argument is even or odd.
- Check if a path argument is a file, a directory or does not exist.
- Write a menu with `case` that prints different messages for `start`, `stop` and anything else.

MISSION 3 | +50 XP

Repeat: for, while and until

Goal: Run the same steps for many items or until a condition changes.

```
for i in 1 2 3 4 5; do
    echo "Number $i"
done

for i in {1..5}; do echo $i; done           # brace range
for i in $(seq 0 5 20); do echo $i; done    # 0 5 10 15 20
for ((i=0; i<3; i++)); do echo "i=$i"; done # C-style loop
```

Looping over files

```
for f in *.txt; do
    echo "Processing $f"
    wc -l "$f"
done

for f in *.jpg; do mv "$f" "photo_$f"; done   # bulk rename
for user in $(cut -d: -f1 /etc/passwd | head -3); do echo "User: $user"; done
```

while and until

```
count=1
while [ $count -le 3 ]; do
    echo "Attempt $count"
    ((count++))
done

until ping -c1 -W1 example.com > /dev/null 2>&1; do
    echo "Waiting for network..."; sleep 2
done
```

Reading a file line by line

```
while IFS= read -r line; do
    echo "Line: $line"
done < people.txt

while IFS=, read -r name dept salary city; do
    echo "$name works in $dept"
done < people.csv
```

break and continue

```
for n in 1 2 3 4 5 6; do
    [ $n -eq 2 ] && continue     # skip 2
    [ $n -eq 5 ] && break        # stop at 5
    echo $n
done                              # prints 1 3 4
```

Warning
Always quote `"$f"` in loops. A filename containing a space will be split into two words otherwise.

Practice

- Print a multiplication table for a number given as an argument.
- Loop over all `.log` files and print the number of lines of each.
- Write a `while` loop that counts down from 5 to 1 and then prints `Liftoff!`.

MISSION 4 | +50 XP

Reuse Code: Functions and Arrays

Goal: Group logic into reusable pieces and work with lists of values.

```
greet() {
    local name="$1"                # local keeps the variable inside
    echo "Hello, $name!"
}
greet "Haani"
greet "Tux"

add() {
    echo $(( $1 + $2 ))              # 'return' value via output
}
result=$(add 4 5)
echo "Sum: $result"

is_even() {
    (( $1 % 2 == 0 ))                # status code is the result
}
is_even 10 && echo "even"
```

Concept	Notes
Arguments	Inside a function, `$1` `$2` `$@` refer to the function's arguments
`return n`	Sets exit status 0 to 255, not text
`echo`	Use it to give back text, capture with `$( ... )`
`local`	Prevents leaking variables into the script

Arrays

```
fruits=("apple" "banana" "cherry")
echo "${fruits[0]}"            # apple
echo "${fruits[@]}"            # all items
echo "${#fruits[@]}"           # length: 3
fruits+=("mango")              # append
for f in "${fruits[@]}"; do echo "- $f"; done

declare -A age                  # associative array (Bash 4+)
age[alice]=30; age[bob]=25
for k in "${!age[@]}"; do echo "$k is ${age[$k]}"; done
```

Sharing code

```
# lib.sh
log() { echo "[$(date +%T)] $*"; }

# main.sh
source ./lib.sh
log "Backup started"
```

Practice

- Write a function `square` that prints the square of its argument.
- Store five names in an array, then print them numbered.
- Move a logging function into its own file and `source` it from another script.

MISSION 5 | +50 XP

Real Scripts: Exit Codes and Safety

Goal: Write scripts that fail loudly, clean up after themselves and are safe to automate.

Exit codes

Every command returns a number: **0 means success**, anything else is an error. Scripts should do the same.

```
cp a.txt /backup/ || { echo "Copy failed" >&2; exit 1; }
if grep -q error app.log; then echo "Errors found"; fi
echo "Last status: $?"
```

Strict mode

```
#!/bin/bash
set -euo pipefail
# -e  stop on first failing command
# -u  error on undefined variables
# -o pipefail  a pipeline fails if any step fails
```

Cleanup with trap

```
tmp=$(mktemp)
trap 'rm -f "$tmp"; echo "cleaned up"' EXIT      # runs however the script ends
trap 'echo "Interrupted"; exit 130' INT
```

Options with getopts

```
while getopts "n:v" opt; do
    case $opt in
        n) name=$OPTARG ;;
        v) verbose=1 ;;
        *) echo "Usage: $0 [-n name] [-v]" >&2; exit 2 ;;
    esac
done
```

Complete example: backup script

```
#!/bin/bash
set -euo pipefail
src="${1:?Usage: $0 SOURCE_DIR}"
dest="$HOME/backups"
stamp=$(date +%Y%m%d_%H%M%S)
[ -d "$src" ] || { echo "No such directory: $src" >&2; exit 1; }
mkdir -p "$dest"
tar -czf "$dest/backup_$stamp.tar.gz" -C "$src" .
echo "Backup saved to $dest/backup_$stamp.tar.gz"
find "$dest" -name 'backup_*.tar.gz' -mtime +7 -delete   # keep one week
```

Tip
Run `shellcheck script.sh` (install with apt) to catch quoting and logic mistakes automatically.

Practice

- Add strict mode and an argument check to an earlier script.
- Use `trap` to delete a temp file even when you press Ctrl+C.
- Run the backup script on a test folder and verify the archive with `tar -tzf`.

BOSS BATTLE | +150 XP

Boss: The Automation Master

Goal: combine everything from this module without hints.

Scenario

Build a single script `toolkit.sh` that reports on a directory, with safe input checks, a function, a loop and a menu.

Tasks

1. Add the shebang and `set -euo pipefail`; make it executable.
2. Accept a directory as `$1`, defaulting to the current folder; exit with an error message if it is not a directory.
3. Write a function `count_files` that prints the number of regular files in the directory.
4. Loop over every `.txt` file and print its name with its line count.
5. Show a `case` menu (`count`, `list`, `quit`) driven by `read`.
6. Store all `.txt` filenames in an array and print how many there are.
7. Print a final message to stderr if no `.txt` files exist and exit with code 3.
8. Run `bash -n` and `bash -x` on your script.

Sample solution

```
#!/bin/bash
set -euo pipefail
dir="${1:-.}"
[ -d "$dir" ] || { echo "Not a directory: $dir" >&2; exit 1; }
count_files() { find "$dir" -maxdepth 1 -type f | wc -l; }
shopt -s nullglob
files=("$dir"/*.txt)
for f in "${files[@]}"; do echo "$f: $(wc -l < "$f") lines"; done
echo "Text files: ${#files[@]}"
read -p "Action (count|list|quit): " act
case "$act" in
  count) count_files ;;
  list)  ls "$dir" ;;
  quit)  exit 0 ;;
  *)     echo "Unknown" >&2 ;;
esac
[ ${#files[@]} -gt 0 ] || { echo "No .txt files" >&2; exit 3; }
```

Check Your Understanding

Q1. What does `#!/bin/bash` do?

Q2. How do you assign a variable?

Q3. Difference between single and double quotes?

Q4. What is $??

Q5. What does $# hold?

Q6. Which test checks whether a file exists and is regular?

Q7. How do you write a C-style loop?

Q8. What does `local` do in a function?

Q9. What does `set -e` do?

Q10. How do you capture a command's output in a variable?

Answers

#	Answer
1	Tells the system to run the script with Bash.
2	name=value (no spaces around =).
3	Double quotes expand variables; single quotes are literal.
4	Exit status of the last command.
5	The number of arguments.
6	[-f file]
7	for ((i=0; i<5; i++)); do ... done
8	Limits a variable to that function.
9	Exits the script when a command fails.
10	var=$(command)

Quick Reference Cheat Sheet

Command	Purpose	Example
#!/bin/bash	Shebang	first line
var="x"	Assign	name="Tux"
$(cmd)	Command output	d=$(date)
$1 $@ $# $?	Args and status	echo "$#"
read -p	Prompt for input	read -p "Name: " n
if [ ]; then fi	Condition	[-f f] && echo yes
case ... esac	Multi-way branch	case $x in a) ;; esac

Command	Purpose	Example
for / while	Loops	for f in *.txt; do ...; done
func() { }	Function	hi() { echo $1; }
set -euo pipefail	Safe mode	top of script
trap	Cleanup	trap 'rm -f $t' EXIT

Up next: Module 08
System Administration. Manage services, logs, scheduled jobs and the firewall like a sysadmin.